JAVA PROGRAMMING PRACTICES

Bank Interest and Dynamic Polymorphism

Basic Information

Week	Week 7
Source file	BankInterestPolymorphism.java
Concepts	Inheritance, overriding, dynamic binding, runtime polymorphism.

Objective / Aim

To implement bank interest and dynamic polymorphism using the concepts listed in the R23 Java Programming Practices syllabus.

System Requirements

JDK (Java Development Kit) and a text editor or Java IDE.

Algorithm / Logic Description

1. Define RBI with an interest-rate method.
2. Override the method in SBI and ICICI.
3. Use an RBI reference to call the selected object's method.

UML / Program Structure

```
RBI
  ▲       ▲
SBI      ICICI
getInterestRate() overridden
```

Source Code

```
import java.util.Scanner;
class RBI
{
    double getInterestRate()
    {
        return 4.0;
    }
}
class SBI extends RBI
{
    @Override double getInterestRate()
    {
        return 7.0;
    }
}
class ICICI extends RBI
{
    @Override double getInterestRate()
    {
        return 6.5;
    }
}
public class BankInterestPolymorphism
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter bank name (RBI/SBI/ICICI): ");
        String name = sc.nextLine().trim().toUpperCase();
        RBI bank;
        if (name.equals("SBI")) bank = new SBI();
        else if (name.equals("ICICI")) bank = new ICICI();
        else bank = new RBI();
        System.out.println(name + " rate of interest is: " + bank.getInterestRate() + "%");
        sc.close();
    }
}
```

Compilation and Execution

```
javac BankInterestPolymorphism.java
java BankInterestPolymorphism
```

Output Screen

```
Enter bank name (RBI/SBI/ICICI): SBI rate of interest is: 7.0%
```

Result

The program demonstrates the listed syllabus concepts. The sample output is based on a compiled execution.